

HelixTerminator

HelixTerminator

Die Zero-Trust-Terminalplattform für Teams - jede SSH-Sitzung gesichert, geteilt und AI-gestützt.

Zusammenfassung

HelixTerminator ist eine Unternehmens-Terminal- und SSH-Sitzungsmanagementplattform, die als Go-Microservices-System mit plattformübergreifenden Flutter-Clients aufgebaut ist. Sie vermittelt, protokolliert und sichert Remote-Sitzungen nach dem Zero-Trust-Modell, ermöglicht Echtzeit-Kollaboration und integriert AI-Unterstützung direkt im Terminal.

Kurzbeschreibung

HelixTerminator ist eine Zero-Trust-Unternehmensplattform für Terminals und SSH-Management: ein Go-Microservices-Backend mit Flutter-Clients für sechs Plattformen. Sie verwaltet Hosts, vermittelt Verbindungen, zeichnet Sitzungen auf, ermöglicht Echtzeit-Kollaboration und bietet AI-gestützte Befehlshilfe, Ausgabenerklärung und Incident-Response.

Ausführliche Beschreibung

HelixTerminator ist eine Terminal- und Remote-Zugriffsplattform für Unternehmen, die aus zwei Modulen besteht - einer Terminal-Plattform und einem Verbindungsbroker - und als Katalog von Go-Microservices mit einem einzigen Flutter-Client für sechs Plattformen implementiert ist. Ihr Ziel ist es, improvisierte SSH-Tools vollständig abzulösen: keine individuellen Laptop-Clients mehr, kein Wildwuchs privater Schlüssel und keine Lücken in der Protokollierung, sondern ein einziges, kontrolliertes, auditierbares und kollaboratives System, das Remote-Zugriff als Infrastruktur behandelt - nicht als persönliche Gewohnheit.

Das Backend steuert den gesamten Lebenszyklus des Remote-Zugriffs von Anfang bis Ende. Hosts und Gruppen werden mit Bastion-/Jump-Host-Ketten verwaltet; ein SSH-Proxy vermittelt Authentifizierung per Passwort, Public Key und Zertifikat; ein Terminal-I/O-Proxy streamt die Sitzung über WebSocket; SFTP ermöglicht wiederaufnehmbare Dateiübertragungen; und es gibt Port-Forwarding, Snippet- und Workspace-Management sowie Sitzungsaufzeichnungen, die als signierte Asciiinema-Wiedergaben gespeichert werden und vertrauenswürdig nachvollziehbar sind. Sicherheit ist von Grund auf nach dem Zero-Trust-Prinzip konzipiert, nicht nachträglich hinzugefügt: Ein Vault bietet geheimnisgeschützte Speicherung ohne Kenntnis der Inhalte, ein PKI-Dienst stellt kurzlebige SSH-Zertifikate aus, sodass keine dauerhaften Anmeldedaten im Umlauf sind, die gestohlen werden könnten; hardwaregestützte Schlüsselketten (Secure Enclave / Android Keystore / DPAPI / HSM) halten Schlüssel vom Datenträger fern; FIDO2/WebAuthn und OIDC/SAML bilden die Authentifizierungsfront; und ein nur anhängbares, Merkle-verkettetes Audit-Log erzeugt manipulationssichere Nachweise für SOC 2 / ISO 27001. Darüber hinaus ermöglicht Echtzeit-Kollaboration mehreren Benutzern, eine Live-Sitzung in den Rollen Beobachter, Co-Pilot oder Besitzer zu teilen, wobei die Konsistenz durch CRDT-Puffer-Synchronisation gewährleistet wird.

Ein AI-Dienst läuft direkt im Terminal und bietet Befehlsautovervollständigung, verständliche Erklärungen von Ausgaben, Anomalieerkennung, Runbook-Generierung und praktische Unterstützung bei Incidents – und verwandelt das Terminal so von einer simplen Schnittstelle in einen Assistenten genau in den entscheidenden Momenten. Die gesamte Plattform ist container-nativ – Kubernetes, Helm, Terraform und ein vollständiger Observability-Stack mit OpenTelemetry, Grafana, Jaeger und Loki – und lässt sich über eine HelixTrack-Brücke und einen lokalen HelixLLM in die größere Helix-Familie einbinden. Alles läuft unter dem Helix-Constitution mit Anti-Bluff-Vererbungsprüfungsmechanismen.

Warum wir es entwickelt haben

Teams verwalten ihre Remote-Infrastruktur über verstreute SSH-Clients ohne gemeinsame Prüfprotokolle, ohne einheitliche Geheimnisverwaltung und ohne Möglichkeit, live bei einem Vorfall zusammenzuarbeiten. HelixTerminator wurde entwickelt, um Remote-Zugriff zu einer kontrollierten, Zero-Trust- und teamorientierten Plattform zu machen – statt zu einem Werkzeug, das an jedem Laptop hängt.

Warum es die Spielregeln ändert

Es bündelt eine ganze Beschaffungsliste in einer einzigen Plattform. Der SSH-Client, der Secrets-Tresor, die Bastion-/PKI-Schicht, Sitzungsaufzeichnungen, Compliance-Audits und Live-Kollaboration sind Dinge, die Teams normalerweise einzeln einkaufen, zusammenflicken und mühsam abstimmen – jedes mit eigenen Lücken an den Schnittstellen. HelixTerminator liefert sie als ein einziges, kontrolliertes System aus und geht noch einen Schritt weiter: Es legt eine AI-Schicht direkt über das Terminal, die unbekannte Ausgaben erklärt und Runbooks *während eines laufenden Vorfalls* erstellt. Was früher undenkbar war, ist jetzt Realität: eine Remote-Zugriffssitzung, die gleichzeitig Zero-Trust-gesichert, manipulationssicher aufgezeichnet, live mit mehreren Bedienern geteilt *und* AI-unterstützt ist – alles auf einen Blick, aus einem einzigen Interface.

Was innovativ ist

- **Dual-Modul-Design** (Terminal-Plattform + Verbindungsbroker), koordiniert über ein Service-Registry, sodass Plattform und Broker-Schicht unabhängig skalieren und weiterentwickelt werden können.
- **Zero-Trust-Sicherheit** durchgängig: kurzlebige SSH-Zertifikate, die von einer PKI ausgestellt werden, ein Zero-Knowledge-Tresor, hardwaregestützte Schlüsselbunde und ein Merkle-verkettetes Prüfprotokoll – keine dauerhaften Anmeldedaten, keine nicht nachvollziehbare Spur.
- **Echtzeit-Kollaboration in Sitzungen** mit CRDT-Puffer-Synchronisation und klar definierten Rollen (Beobachter / Co-Pilot / Besitzer), sodass mehrere Bediener gleichzeitig an einem Terminal arbeiten können, ohne sich gegenseitig zu behindern.
- **AI-gestützte Operationen**, die direkt in das Live-Terminal integriert sind: Autovervollständigung, Erklärung von Ausgaben, Anomalieerkennung sowie Runbook- und Vorfallunterstützung genau dort, wo sie gebraucht werden.
- **Plattformübergreifender Flutter-Client**, der sechs Plattformen aus einer einzigen Codebasis bedient, sodass Desktop-, Mobile- und Web-Erlebnis nahtlos synchron bleiben.

Größte technische Herausforderungen & wie wir sie gelöst haben

- **Sicheren Remote-Zugriff ohne dauerhafte Anmeldedaten, die gestohlen werden können** – langlebige Schlüssel sind die klassische Schwachstelle (vector). Gelöst durch einen PKI-Dienst, der kurzlebige SSH-Zertifikate auf Anfrage ausstellt,

einen Zero-Knowledge-Tresor, der Geheimnisse speichert, die der Server selbst nicht lesen kann, und hardwaregestützte Schlüsselspeicher (Secure Enclave / Android Keystore / DPAPI / HSM), sodass private Daten niemals ungeschützt auf der Festplatte liegen.

- **Mehrere Bediener gleichzeitig an einer Sitzung arbeiten lassen, ohne den Puffer zu beschädigen** – gleichzeitige Änderungen an einem geteilten Terminal sind ein schwieriges Konsistenzproblem. Gelöst durch CRDT-basierte Puffer-Synchronisation, die – anders als Operational Transformation (laut ADR-006) – ohne zentrale Schiedsstelle konvergiert.
- **Compliance-Nachweise manipulationssicher machen** – ein Audit-Log, das bearbeitet werden kann, beweist nichts. Gelöst durch ein nur anhängbares, Merkle-verkettetes Protokoll, bei dem jede Manipulation die Hash-Kette unterbricht und so exportierbare Nachweise für SOC 2 / ISO 27001 / FedRAMP erzeugt.
- **Einheitliches Nutzererlebnis auf Desktop, Mobile und Web ohne drei separate Codebasen** – gelöst durch einen einzigen Flutter-/Dart-Client nach dem BLoC-Muster, wobei Flutter (laut ADR-001) gegenüber Electron bevorzugt wurde, um sechs Plattformen aus einer einzigen Quelle zu bedienen.

Technologie-Stack

- **Go-Microservices** – die Backend-Flotte (SSH-Proxy, Terminal, Vault, PKI, Audit und mehr); ausgewählt wegen ihres Nebenläufigkeitsmodells und des geringen Laufzeit-Footprints, ideal für Dienste, die zahlreiche langlebige Streaming-Sitzungen gleichzeitig verwalten (ADR-002: Go gegenüber Rust/Node).
- **Flutter / Dart (BLoC)** – ein einziger Client-Code für sechs Plattformen, wobei BLoC für vorhersehbaren Zustand sorgt; Flutter wurde gegenüber Electron gewählt (ADR-001), um die Wartung separater nativer und Web-Frontends zu vermeiden.
- **PostgreSQL** – der primäre Datenspeicher, ausgewählt gegenüber CockroachDB (ADR-004) wegen eines ausgereiften, gut verstandenen transaktionalen Kerns.
- **Kafka + RabbitMQ** – die Nachrichten- und Streaming-Schicht für Sitzungssegmente und Ereignisse (ADR-003), die ein dauerhaftes Log mit flexibler Warteschlangenverwaltung kombiniert.
- **Redis** – speichert Terminal-Scrollback-Puffer und heißen Sitzungsstatus, wo niedrige Latenz wichtiger ist als Dauerhaftigkeit.
- **SPIFFE/SPIRE + mTLS** – vergibt kryptografische Arbeitslast-Identitäten (ADR-005), sodass der Dienst-zu-Dienst-Verkehr gegenseitig authentifiziert wird und Zero Trust innerhalb des Mesh – nicht nur am Rand – umgesetzt wird.

- **Ed25519 (EdDSA)** – signiert JWTs und Sitzungsaufzeichnungen (ADR-009) und ermöglicht so schnelle, moderne Signaturen, die aufgezeichnete Sitzungen überprüfbar machen.
- **Kubernetes + Helm + Terraform** – containerbasierte Bereitstellung mit reproduzierbarer, versionskontrollierter Infrastruktur (ADR-007/008).
- **OpenTelemetry, Grafana, Jaeger, Loki** – der Observability-Stack für Traces, Metriken, Dashboards und Logs; **Falco, Trivy, Cosign, Sealed Secrets** – Laufzeit-Bedrohungserkennung, Image-Scanning, Artefakt-Signierung und verschlüsselte Geheimnisübermittlung über die gesamte Lieferkette.

Status & Transparenzhinweise

- **Status: Beta.** Ein umfangreicher, aktiv weiterentwickelter Codebase (erstellt am 04.07.2026). Die numerischen Spezifikationsangaben im MVP-Forschungspaket des Projekts (Endpunkt-, Tabellen- und Dienstanzahlen) sind Entwurfsziele aus docs/research/mvp/ und nicht als vollständig implementiert bestätigt. Sie werden daher oben als Architekturumfang und nicht als ausgelieferte Metriken dargestellt. Latenz-/SLO-Angaben und „produktionsreif“-Behauptungen sind nicht unabhängig verifiziert.
- **Lizenz: Apache-2.0** (gemäß GitHub API).

Prioritätsstufe: Helix-primär.